Capstone Submission Report

Usage Metering & Billing Engine · Backend Engineering Track

Author: Rishabh Ahuja

Repository: flyrank-capstone-metering-billing

Evaluation Date: November 2026

Stack: Python 3.10 · FastAPI · SQLAlchemy · SQLite/Docker · Stripe Test Mode

1. Executive Summary & Architecture

The **Usage Metering & Billing Engine** is a production-hardened microservice designed to resolve the three existential problems in multi-tenant SaaS: **accurate usage tracking**, **real-time quota enforcement**, and **zero-float financial calculations**. Constructed with strict layered architecture, the service separates HTTP transport concerns from atomic domain business logic and persistence. Guaranteed exactly-once ingestion eliminates double charging under network retries, while pre-action boundary guards protect revenue and service reliability.

2. Acceptance Probes & Empirical Verification Record

All 5 behavioral acceptance probes defined in Section 12 of the capstone brief have been implemented and verified:

Probe	Target Behavioral Invariant	Status & Empirical Proof
Probe 1: Metering Idempotency	Identical billable request sent twice with same idempotency key records exactly once; retry mirrors first response.	**PASS** 10 calls logged, 0 duplicate events, 200 OK replay
Probe 2: Boundary Honesty	Drive tenant to exact quota (1,000/1,000). Boundary call accepted; call 1,001 returns HTTP 429 with Retry-After. Lapsed returns 402.	**PASS** 429 API_CALL_QUOTA_EXCEEDED returned on call 1001
Probe 3: Stripe Checkout Sync	Checkout completed webhook flips tenant Free to Pro; GET /usage reflects new limits (100k calls, 10M tokens).	**PASS** Sub upgraded to Pro, limits expanded dynamically
Probe 4: Webhook Security	Forged webhook signature rejected with HTTP 400. Replayed webhook acknowledged cleanly without duplicate processing.	**PASS** HMAC-SHA256 verified; replay ignored safely
Probe 5: Token Pricing Rules	Cached tokens discounted; reasoning tokens billed as output; micro-cent integer math with ceil conversion to integer cents.	**PASS** 3610 microcents -> 4 cents calculated with zero float drift

3. Empirical Automated Test Suite Output

```
tests/test_billing_engine.py::test_probe_1_idempotent_event_deduplication PASSED [ 10%]
tests/test_billing_engine.py::test_probe_1_header_idempotency_key_override PASSED [ 20%]
tests/test_billing_engine.py::test_probe_2_exact_quota_boundary_and_exhaustion PASSED [ 30%]
tests/test_billing_engine.py::test_probe_2_lapsed_or_unpaid_tenant_returns_402 PASSED [ 40%]
tests/test_billing_engine.py::test_probe_3_checkout_and_webhook_flips_free_to_pro PASSED [ 50%]
tests/test_billing_engine.py::test_probe_4_forged_webhook_rejected_with_400 PASSED [ 60%]
tests/test_billing_engine.py::test_probe_4_webhook_replay_deduplication PASSED [ 70%]
tests/test_billing_engine.py::test_probe_5_token_pricing_rules PASSED [ 80%]
tests/test_billing_engine.py::test_billable_generate_endpoint PASSED [ 90%]
tests/test_billing_engine.py::test_invoice_generation PASSED [100%]
============================== 10 passed in 1.25s ==============================
```

4. Architectural Implementation Highlights

• **1. Zero-Float Money Math:** Internal pricing uses integer micro-cents scaled by 1,000. Calculations are exact: input (75 mc/1k), cached input (38 mc/1k - 50% discount), and output + reasoning (300 mc/1k). Cents conversion applies math.ceil to prevent fractional revenue loss.

• **2. Atomic Concurrency Guard:** Metering leverages a database composite unique constraint on (tenant_id, idempotency_key). Concurrent duplicate requests are safely resolved by catching database integrity collisions and returning the cached response.

• **3. Cryptographic Webhook Security:** Stripe webhook signatures are validated using constant-time HMAC-SHA256 comparison with a 300-second timestamp freshness tolerance to neutralize timing attacks and replay forgeries.

• **4. Asynchronous Background Reconciliation:** An autonomous background reconciliation thread continuously audits tenant subscription statuses against billing records to resolve out-of-band drifts.

• **5. Repo Hygiene & Secrets Management:** Zero secrets committed. All credentials use environment variables with a sanitized .env.example template.

5. Submission Checklist & Deliverables

File / Requirement	Description & Purpose	Verification Status
`capstone.yaml`	Evaluation manifest specifying run, seed, test, and probed endpoints	**Verified**
`EVIDENCE.md`	Pasted terminal transcripts, curl traces, and mathematical proofs per probe	**Verified**
`BUILDLOG.md`	Transparent AI-assistance log detailing challenges, corrections, and ownership	**Verified**
`README.md`	Complete system architecture, data model, setup guide, and honest limitations	**Verified**
`.env.example`	Sanitized environment configuration template with zero exposed secrets	**Verified**